



Siete modelos de concurrency

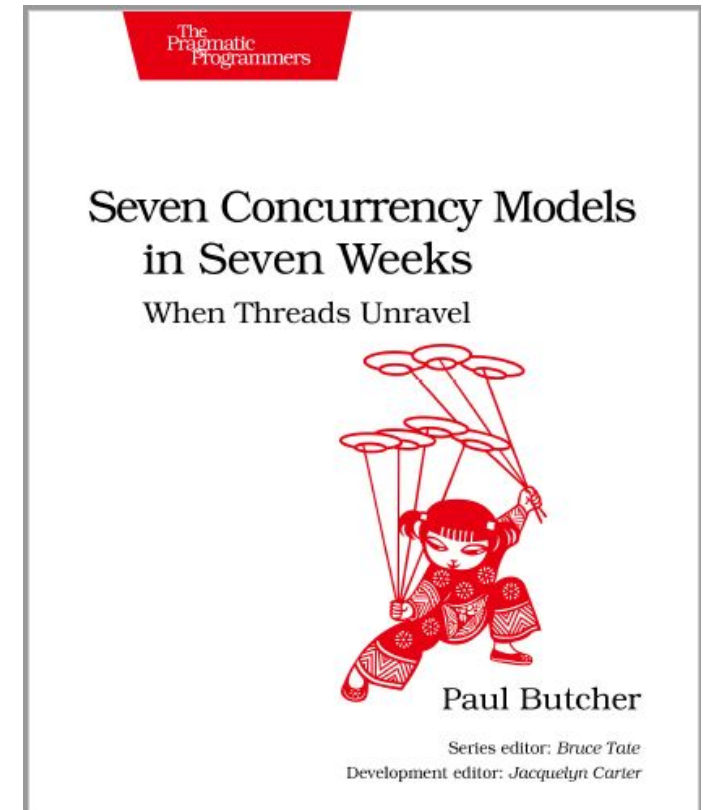
Capitulo 2 día 1

*Hilos y cerrojos (Threads
and locks)*

Seven Concurrency Models

- *Introduction*
- *Threads and locks*
- *Functional programming*
- *The Clojure Way—separating identity and state*
- *Actors*
- *Communicating Sequential Processes*
- *Data parallelism*
- *The Lambda Architecture*

2014



Hilos y cerrojos (Threads and locks)

- La programación con hilos y cerrojos es como un vehículo Ford modelo T. Puede llevarnos del punto A al punto B, pero es primitivo, difícil de conducir, poco confiable –incluso peligroso- comparado con nuevas tecnologías.
- Olvidando un poco sus problemas, la programación con hilos y cerrojos aún se utilizan para escribir mucho software concurrente y como soporta otras tecnologías concurrentes, es bueno estudiarla.



Algo muy simple

- Los hilos y los cerrojos (Locks) son poco más que una formalización de lo que subyace el hardware realmente lo hace. Esa es su gran fuerza y, a la vez, su gran debilidad.
- Debido a que son tan simples, casi todos los lenguajes los admiten de una forma u otra, e imponen muy pocas restricciones sobre lo que se puede lograr a través de su uso.
- Pero tampoco proporcionan casi ninguna ayuda al pobre programador, haciendo que los programas sean muy difíciles de acertar en primer lugar e incluso más difícil de mantener.
- En el capítulo usan el lenguaje Java
 - Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java concurrency in practice*. Addison-Wesley Professional.
 - Urma, R.-G., Fusco, M., & Mycroft, A. (2018). *Modern Java in action: Lambdas, streams, functional and reactive programming*. Manning Publications.

Actividades propuestas para cada día

- Day 1: Mutual Exclusion and Memory Models
 - [Día 1: Exclusión mutua y modelos de memoria]
- Day 2: Beyond Intrinsic Locks
 - [Día 2: Más allá de los cerrojos intrínsecos]
- Day 3: On the Shoulders of Giants
 - [Día 3: Sobre los hombros de gigantes]



Día 1: Exclusión mutua y modelos de memoria

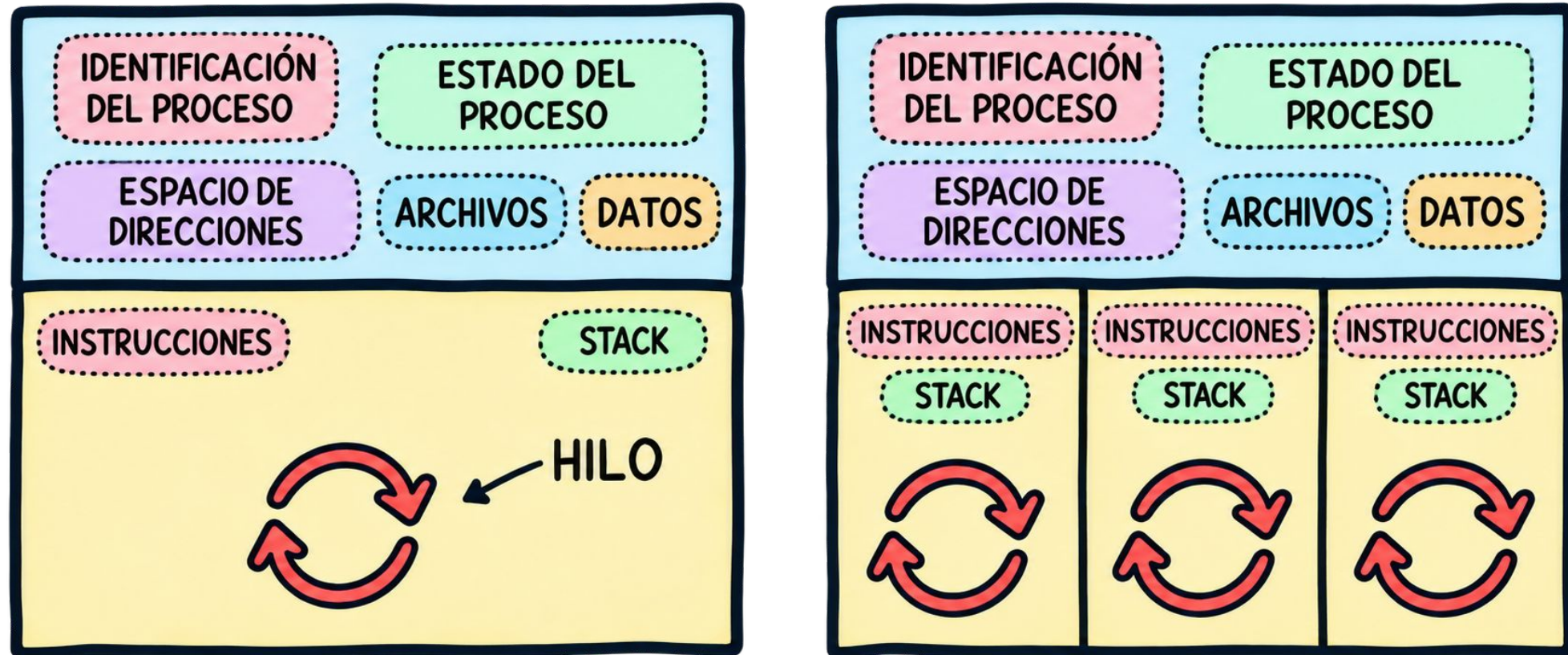
- Si ha realizado alguna programación concurrente, probablemente ya esté familiarizado con el concepto de exclusión mutua, utilizando cerrojos/locks para garantizar que solo un hilo puede acceder a los datos a la vez. Y también estará familiarizado con formas en las que la exclusión mutua puede salir mal, incluidas las condiciones de carrera e interbloqueos –deadlocks- (no se preocupe si estos términos no significan nada para usted todavía, cubriremos todos muy pronto).
- Estos son problemas reales y pasaremos mucho tiempo hablando de ellos pero resulta que hay algo aún más básico de lo que debemos preocuparnos cuando se trata de la memoria compartida: el modelo de memoria...
- ...Y las condiciones de carrera y los puntos muertos [deadlock] pueden causar un comportamiento extraño. La memoria compartida también es “extraña”.
- Sin embargo, nos estamos adelantando a nosotros mismos. Empecemos por estudiar cómo crear un hilo (thread).

Procesos con un hilo y con multiples hilos



Revisar los capítulos 3 y 4 del libro *Operating System Concepts* de **Silberschatz, A. Galvin P.B. y Gagne, G.** Wiley.

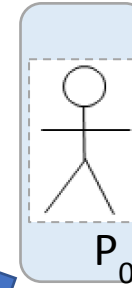
Procesos con un hilo y con multiples hilos



Creando un hilo

- La unidad básica de concurrencia en Java es el hilo, que, como su nombre indica, encapsula un solo hilo de control. Los hilos se comunican entre sí a través de la *memoria compartida*.
- Ningún libro de programación está completo sin un ejemplo de "¡Hola, mundo!" entonces sin más preámbulos, aquí hay una versión multiproceso de "Hola mundo" [revisar antes lo del "Código del libro de los siete modelos de concurrencia"]:
 - HelloWorld.java

```
public class HelloWorld {
```



```
    public static void main(String[] args) throws InterruptedException {  
        Thread myThread = new Thread() {  
            public void run() {  
                System.out.println("Hello from new thread");  
            }  
        };  
  
        myThread.start();  
        Thread.yield();  
        System.out.println("Hello from main thread");  
        myThread.join();  
    }  
}
```

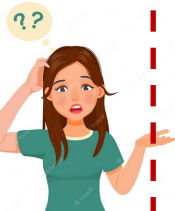


Creando un hilo

- Este código crea una instancia de Thread [hilo] y luego la inicia. A partir de ese punto, el método run() del hilo se puede ejecutar al mismo tiempo que el resto de main ().

- Finalmente, join() espera a que termine el hilo (lo que sucede cuando run() retorne) Cuando ejecuta esto, puede obtener este resultado:

- Hello from main thread
- Hello from new thread



- O también puede obtenerse
 - Hello from new thread
 - Hello from main thread



```
public class HelloWorld {
```

```
    public static void main(String[] args) throws InterruptedException {
```

```
        Thread myThread = new Thread() {
```

```
            public void run() {
```

```
                System.out.println("Hello from new thread");
```

```
            }
```

```
        };
```

```
        myThread.start();
```

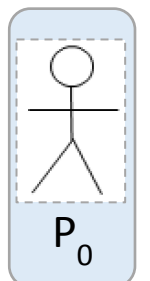
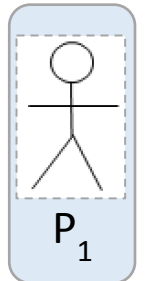
```
        Thread.yield();
```

```
        System.out.println("Hello from main thread");
```

```
        myThread.join();
```

```
    }
```

```
}
```



Creando un hilo

- Este tipo de dependencia del tiempo es una de las cosas que dificultan la programación multiproceso; el hecho de que vea un comportamiento una vez que ejecuta su código no significa que lo verá de manera consistente después.
- El código de “Hello, World!” multihilo tiene la siguiente línea:
 - `Thread.yield();`
- De acuerdo con la documentación en java `yield()` es:
 - una pista para el planificador de que el hilo actual está dispuesto a ceder su uso actual de un procesador.
- Sin esta llamada, la sobrecarga de inicio del nuevo hilo significaría que es casi seguro que el hilo principal llegue primero a su `println()` (aunque no se garantiza que este sea el caso, y como veremos, en la programación concurrente si algo puede suceder, luego tarde o temprano sucederá, probablemente en el momento más inconveniente).
- Intente comentar este método y vea qué sucede. ¿Qué sucede si lo cambia a `Thread.sleep(1)`?

Nuestra primera cerrojo/candado [lock]

- Cuando varios subprocesos/hilos acceden a la memoria compartida, pueden terminar pisándose los dedos entre ellos. Evitamos esto mediante la ***exclusión mutua*** a través de cerrojos/candados [locks], que pueden ser retenidos [usados] por un solo hilo a la vez.
- Creemos un par de hilos que interactúen entre sí: [ver el código Counting.java]
- En ese código se tiene una clase Counter muy simple y dos hilos, cada uno de los cuales llama a su método increment() 10,000 veces. Muy simple -y muy mal hecho-.
- Intente ejecutar este código y obtendrá una respuesta diferente cada vez. Las últimas tres veces que lo ejecuté obtuve 13850, 11867, luego 12616. El motivo es una condición de carrera (comportamiento que depende del tiempo relativo de las operaciones) en el uso de los dos subprocesos/hilos del miembro de recuento de Counter.
- La solución es *sincronizar el acceso* para la variable count.

Nuestro primer cerrojo/candado [lock]

- Cuando varios subprocesos/hilos acceden a la memoria compartida, pueden terminar pisándose los dedos entre ellos. Evitamos esto mediante la exclusión mutua a través de cerrojos/candados [locks], que pueden ser retenidos [usados] por un solo hilo a la vez.
- Creemos un par de hilos que interactúen entre sí: [ver el código Counting.java]

```
public class Counting {  
    public static void main(String[] args) throws InterruptedException {  
        class Counter {  
            private int count = 0;  
            public void increment() { ++count; }  
            public int getCount() { return count; }  
        }  
        final Counter counter = new Counter();  
        class CountingThread extends Thread {  
            public void run() {  
                for(int x = 0; x < 10000; ++x)  
                    counter.increment();  
            }  
        }  
  
        CountingThread t1 = new CountingThread();  
        CountingThread t2 = new CountingThread();  
        t1.start(); t2.start();  
        t1.join(); t2.join();  
        System.out.println(counter.getCount());  
    }  
}
```

Nuestro primer cerrojo/candado [lock]

- Una forma de hacerlo es utilizar el bloqueo/lock intrínseco que viene integrado en cada objeto Java (a veces lo escuchará llamar mutex, monitor o sección crítica) sincronizando increment():

```
class Counter {  
    private int count = 0;  
    ➤ public synchronized void increment() { ++count; }  
    public int getCount() { return count; }  
}
```

synchronized es egoísta, no se deja hablar [no se puede interrumpir], el paso del tiempo no le importa y es de un solo uso.



- Ahora increment () reclama el lock del objeto Counter cuando se llama y lo libera cuando regresa, por lo que solo un hilo puede ejecutar su cuerpo [el código del hilo] a la vez. Cualquier otro hilo que lo llame se bloqueará [esperará] hasta que el bloqueo se libere (más adelante en este capítulo veremos que, para casos simples como este, en los que solo está involucrada una variable, el paquete java.util.concurrent.atomic proporciona buenas alternativas para usar un cerrojo - lock).
- Efectivamente, cuando ejecutamos esta nueva versión, obtenemos el resultado 20000 cada vez.
- Pero no todo es color de rosa, nuestro nuevo código todavía contiene un error sutil, cuya causa revisaremos a continuación.

“42”

Memoria misteriosa

The answer to life, universe and everything



- <https://www.youtube.com/watch?v=aboZctrHfK8>
- https://es.wikipedia.org/wiki/El_sentido_de_la_vida,_el_universo_y_todo_lo_dem%C3%A1s

Memoria misteriosa

¿Qué generará este código?

```
Line 1 public class Puzzle {  
-     static boolean answerReady = false; ←  
-     static int answer = 0; ←  
-     static Thread t1 = new Thread() {  
5         public void run() {  
-             answer = 42; ←  
-             answerReady = true; ←  
-         }  
-     };  
  
10    static Thread t2 = new Thread() {  
-        public void run() {  
-            if (answerReady)  
-                System.out.println("The meaning of life is: " + answer);  
-            else  
15                System.out.println("I don't know the answer");  
-        }  
-    };  
  
-  
-    public static void main(String[] args) throws InterruptedException {  
20        t1.start(); t2.start();  
-        t1.join(); t2.join();  
-    }  
- }
```

Memoria misteriosa

- Si está pensando en "condición de carrera" Está absolutamente en lo correcto. Podríamos ver la respuesta al significado de la vida o una admisión decepcionante de que nuestro computador no lo sabe, dependiendo del orden en el que se ejecutan los hilos.
- Pero eso no es todo, hay otro resultado que podríamos ver:
The meaning of life is: 0
- ¡¿Qué?! ¿Cómo es posible que la respuesta sea cero si el valor de answerReady es verdadero? Es casi como si algo cambiara las líneas 6 y 7 debajo de nuestros pies.



The meaning of life is 42



I don't know the answer



The meaning of life is 0

Memoria misteriosa

- Bueno, resulta que es posible que haga exactamente eso.
- Varias cosas, de hecho:
 - El compilador puede optimizar estáticamente su código reordenando las cosas.
 - La JVM puede optimizar dinámicamente su código reordenando las cosas.
 - El hardware en el que está ejecutando puede optimizar el rendimiento reordenando las cosas.
- Va más allá de reordenar. A veces, los efectos no se vuelven visibles para otros hilos en absoluto. Imagine que reescribimos run() [del hilo 2] de la siguiente manera:

```
public void run() {  
    while (!answerReady)  
        Thread.sleep(100);  
    System.out.println("The meaning of life is: " + answer);  
}
```

Memoria misteriosa

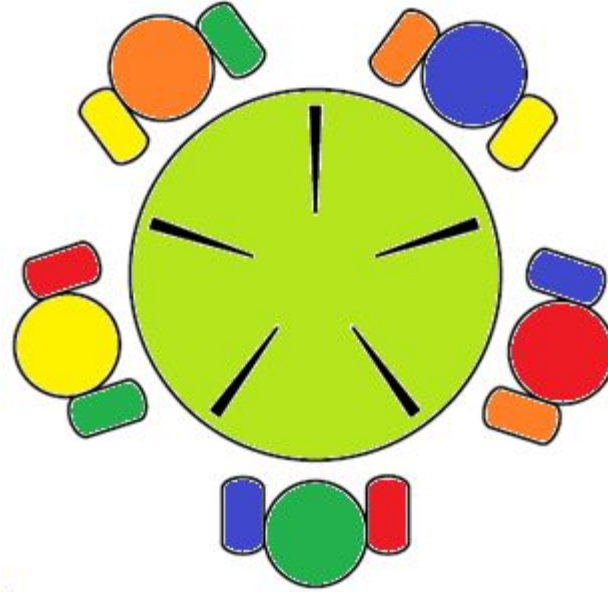
- Es posible que nuestro programa nunca se cierre porque es posible que `answerReady` nunca llegue a tener el valor de verdadero.
- Si su primera reacción a esto es que el compilador, la JVM y el hardware deben mantener sus dedos pegajosos fuera de su código, es comprensible.
- Lamentablemente, también es inalcanzable: gran parte del mayor rendimiento que hemos visto en los últimos años se debe exactamente a estas optimizaciones. Los computadores paralelos de memoria compartida, en particular, dependen de ellos. Así que estamos atrapados en tener que lidiar con las consecuencias.
- Claramente, esto no puede ser un evento gratuito; necesitamos algo que nos diga en qué podemos y no podemos confiar. Ahí es donde entra en juego el modelo de memoria de Java.

Visibilidad de la memoria

- El modelo de memoria de Java define cuándo los cambios en la memoria realizados por un hilo se vuelven visibles para otro hilo. La conclusión es que no hay garantías a menos que tanto el hilo de lectura como el de escritura utilicen sincronización.
- **Visitar:**
<https://docs.oracle.com/javase/specs/jls/se7/html/jls-17.html#jls-17.4>
- Ya hemos visto un ejemplo de sincronización: obtener el bloqueo/lock intrínseco de un objeto. Otros incluyen iniciar un hilo, detectar que un hilo se detiene con `join()` y usar muchas de las clases en el paquete `java.util.concurrent`.

Visibilidad de la memoria

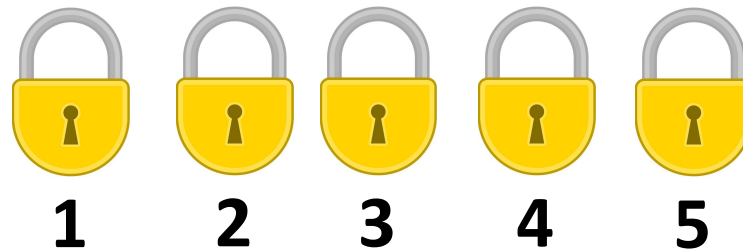
- No basta con sincronizar `increment()`; `getCount()` también debe sincronizarse. Si no es así, un subproceso/hilo que llama a `getCount()` puede ver un valor obsoleto (como sucede, la forma en que se usa `getCount()` en el código de la página 12 [Counting.java] es segura para subprocesos/hilos, porque se llama después de una llamada a `join()`, pero es una bomba de tiempo esperando a cualquiera que use Counter).
- Un punto importante que se pasa por alto fácilmente es que ambos hilos deben usar la sincronización. *No es suficiente que solo el hilo realice cambios para hacerlo.*
- Esta es la causa de un error sutil que aún permanece en el código de la página 13 [synchronized void increment() para counting].
- Hemos hablado sobre las condiciones de carrera y la visibilidad de la memoria, dos formas comunes en las que los programas multiproceso pueden fallar. Ahora pasaremos al tercero: punto muerto [deadlock].



Multiple Locks

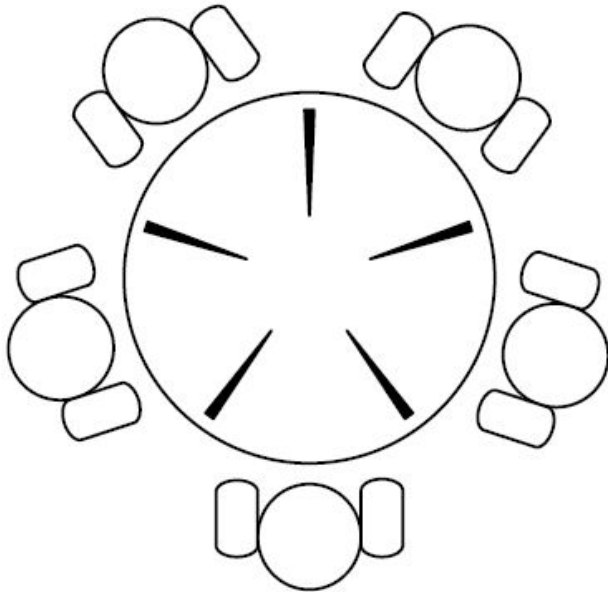
Multiple Locks

- Después de leer el texto anterior, se puede pensar que la única forma de estar seguro en un mundo de múltiples hilos es sincronizar todos los métodos. Desafortunadamente, no es tan fácil.
- En primer lugar, esto sería terriblemente ineficaz. Si todos los métodos estuvieran sincronizados, la mayoría de los subprocesos/hilos probablemente pasarían la mayor parte de su tiempo bloqueados, anulando el punto de hacer que su código sea concurrente en primer lugar. Pero esta es la menor de sus preocupaciones: tan pronto como tenga más de un bloqueo (recuerde, en Java cada objeto tiene su propio bloqueo), crea la oportunidad para que los subprocesos/hilos se bloqueen.



Cena de los filósofos

- Demostraremos el estancamiento/deadlock con un pequeño ejemplo que se usa comúnmente en artículos académicos sobre concurrencia: el problema de la “cena de los filósofos”. Imagínese que cinco filósofos están sentados alrededor de una mesa, con cinco (no diez) palillos colocados así:

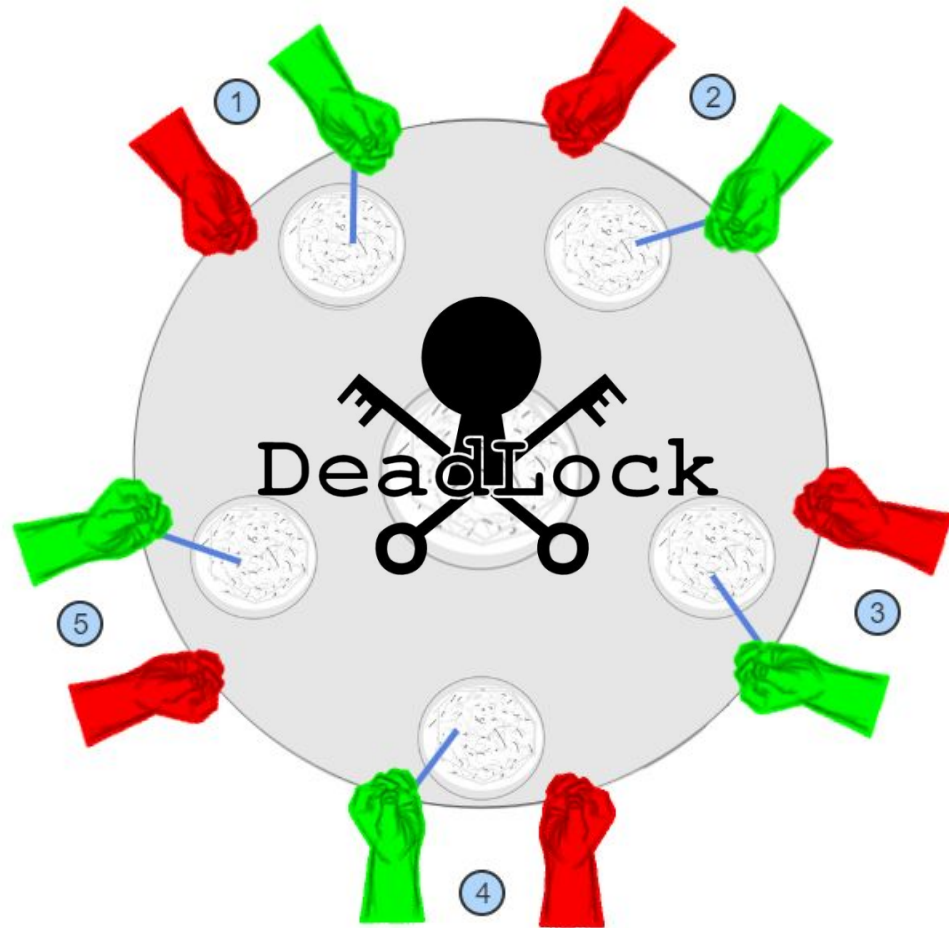


Cena de los filósofos

- Un filósofo está pensando o tiene hambre. Si tiene hambre, toma los palillos a cada lado y come un rato (sí, nuestros filósofos son hombres, las mujeres se comportarían con más sensatez). Cuando termina, los deja.
- Así es como podríamos implementar uno de nuestros filósofos:
 - `Philosopher.java`



La situación de la cena de los filósofos



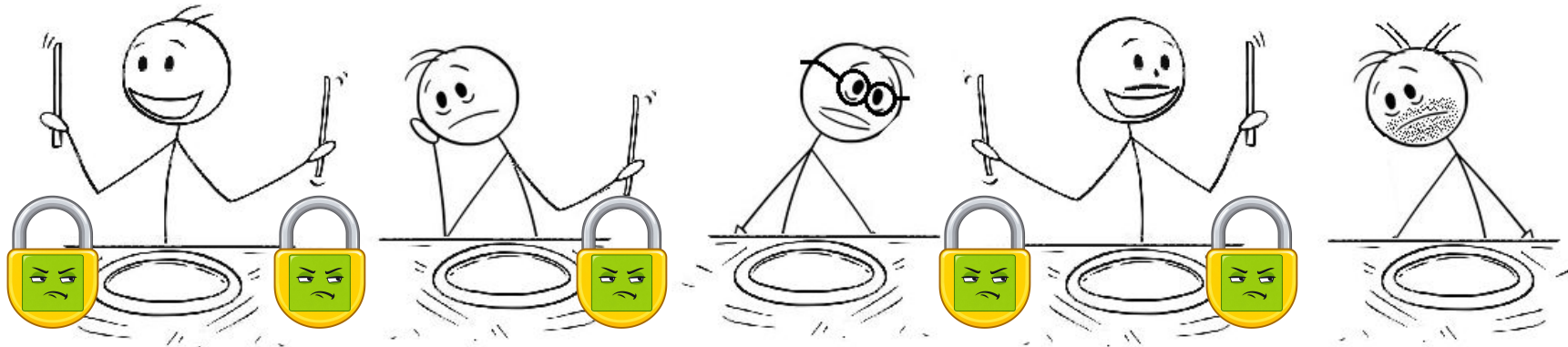
10. Cada uno agarra el palillo izquierdo, pero todos los palillos están asignados, sin dejar ninguno libre para la mano derecha de ningún filósofo. Ningún filósofo puede progresar, por lo que ninguno comerá.

```
Line 1 class Philosopher extends Thread {  
-     private Chopstick left, right;  
-     private Random random;  
-  
5     public Philosopher(Chopstick left, Chopstick right) {  
-         this.left = left; this.right = right;  
-         random = new Random();  
-     }  
-  
10    public void run() {  
-        try {  
-            while(true) {  
-                Thread.sleep(random.nextInt(1000)); // Think for a while  
-                synchronized(left) { // Grab left chopstick //  
15                synchronized(right) { // Grab right chopstick //  
-                    Thread.sleep(random.nextInt(1000)); // Eat for a while  
-                }  
-            }  
-        }  
20    } catch (InterruptedException e) {}  
-    }  
- }
```



Cena de los filósofos

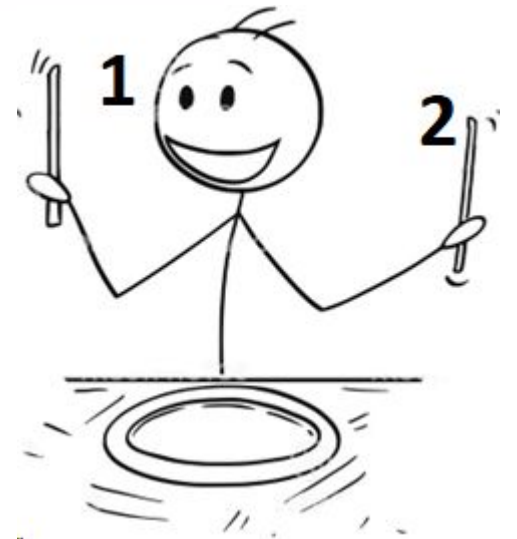
- Las líneas 14 y 15 demuestran una forma alternativa de reclamar el bloqueo intrínseco de un objeto: `synchronized(object)`.
- En mi máquina [del autor del libro], si configuro cinco de estos en funcionamiento simultáneamente, generalmente funcionan muy bien durante horas y horas (mi registro es de más de una semana). Entonces, de repente, todo se detiene.
- Después de pensarlo un poco, es obvio lo que está sucediendo: si todos los filósofos deciden comer al mismo tiempo, todos agarran su palillo izquierdo y luego se quedan atascados, cada uno tiene un palillo y cada uno está bloqueado esperando al filósofo en su Derecha. Punto muerto [deadlock].
- El interbloqueo [deadlock] es un peligro siempre que un hilo intenta retener más de un bloqueo.
- Afortunadamente, existe una regla simple que garantiza que nunca se tendrá deadlock: adquirir siempre bloqueos en un orden global fijo.



```
class Philosopher extends Thread {  
➤ private Chopstick first, second;  
private Random random;
```

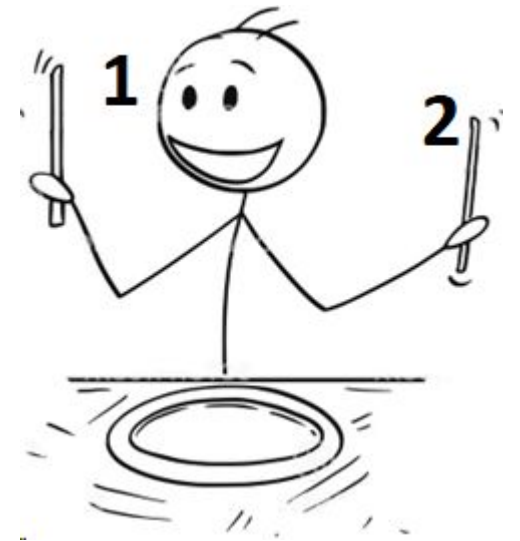
```
public Philosopher(Chopstick left, Chopstick right) {  
➤ if(left.getId() < right.getId()) {  
➤     first = left; second = right;  
➤ } else {  
➤     first = right; second = left;  
➤ }  
    random = new Random();  
}
```

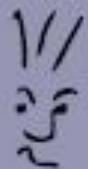
```
public void run() {  
    try {  
        while(true) {  
            Thread.sleep(random.nextInt(1000)); // Think for a while  
➤            synchronized(first) { // Grab first chopstick  
➤                synchronized(second) { // Grab second chopstick  
                    Thread.sleep(random.nextInt(1000)); // Eat for a while  
                }  
            }  
        }  
    } catch (InterruptedException e) {}  
}
```



Cena de los filósofos

- En lugar de aferrarnos a los palillos izquierdo y derecho, ahora nos aferramos al primero y al segundo, usando el miembro de identificación de Chopstick para asegurarnos de bloquear siempre los palillos en orden creciente de identificación (en realidad no nos importa qué identificación tienen los palillos, solo que ellos 'son únicos y ordenados'). Y efectivamente, ahora las cosas funcionarán felizmente para siempre sin bloquearse.
- Es fácil ver cómo cumplir con la regla de pedido global cuando el código para adquirir bloqueos está todo en un solo lugar. Se vuelve mucho más difícil en un programa grande, donde una comprensión global de lo que hace todo el código no es práctica.





Joe asks:

Can I Use an Object's Hash to Order Locks?

One piece of advice you'll often see is to use an object's hash code to order lock acquisition, such as shown here:

```
if(System.identityHashCode(left) < System.identityHashCode(right)) {  
    first = left; second = right;  
} else {  
    first = right; second = left;  
}
```

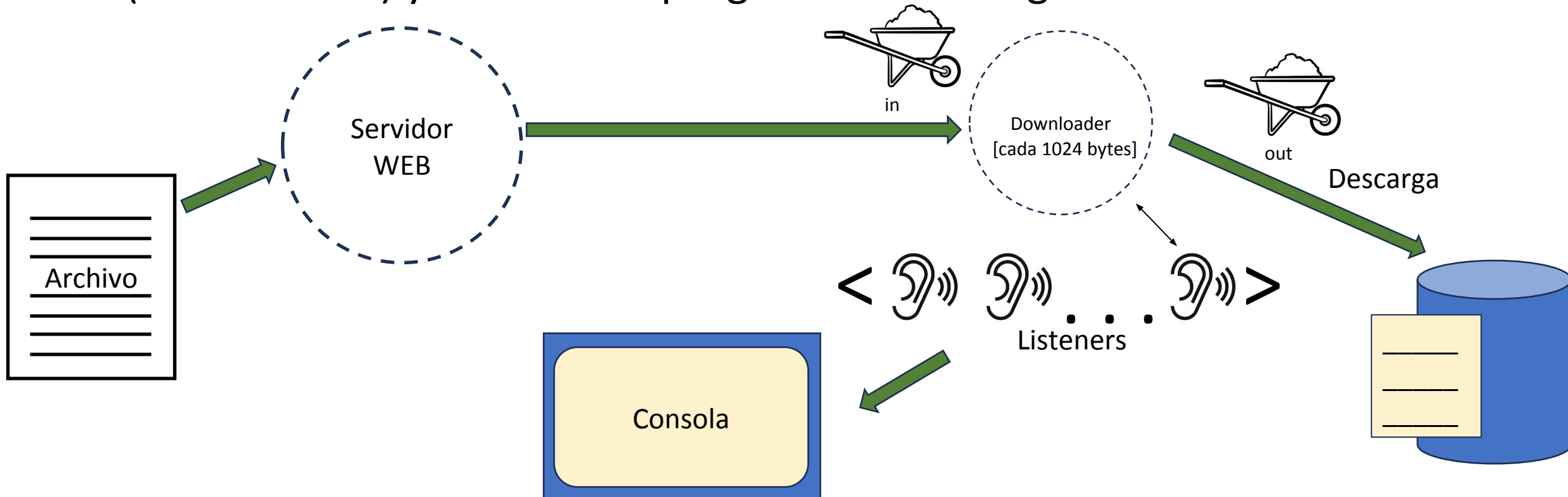
This technique has the advantage of working for any object, and it avoids having to add a means of ordering your objects if they don't already define one. But hash codes aren't guaranteed to be unique (two objects are very unlikely to have the same hash code, but it does happen). So personally speaking, I wouldn't use this approach unless I really had no choice.



Los peligros de los métodos “alienígenas”

Descripción del programa que sigue

- El ejemplo que sigue implementa una aplicación que descarga un archivo desde una URL utilizando un hilo independiente (**Downloader**) y actualiza el progreso de descarga en la consola.



Los peligros de los métodos “alienígenas”



- Los programas grandes a menudo utilizan listeners (oyentes) para desacoplar módulos. Aquí por ejemplo, es una clase que se descarga desde una URL y permite que `aProgressListeners` estar registrado:

```
class Downloader extends Thread {  
    private InputStream in;  
    private OutputStream out;  
    private ArrayList<ProgressListener> listeners;  
  
    public Downloader(URL url, String outputFilename) throws IOException {  
        in = url.openConnection().getInputStream();  
        out = new FileOutputStream(outputFilename);  
        listeners = new ArrayList<ProgressListener>();  
    }  
    public synchronized void addListener(ProgressListener listener) {  
        listeners.add(listener);  
    }  
    public synchronized void removeListener(ProgressListener listener) {  
        listeners.remove(listener);  
    }  
}
```

Los peligros de los métodos “alienígenas”



- Los programas grandes a menudo utilizan listeners (oyentes) para desacoplar módulos. Aquí por ejemplo, es una clase que se descarga desde una URL y permite que `aProgressListeners` estar registrado:

```
private synchronized void updateProgress(int n) {  
    for (ProgressListener listener: listeners)  
        listener.onProgress(n);  
}
```

```
public void run() {  
    int n = 0, total = 0;  
    byte[] buffer = new byte[1024];
```

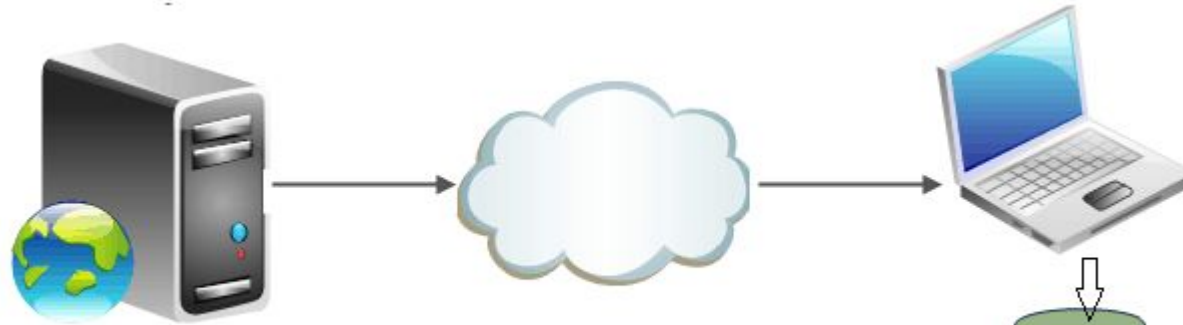
Escribe en el
archivo

```
try {  
    while((n = in.read(buffer)) != -1) {  
        out.write(buffer, 0, n);  
        total += n;  
        updateProgress(total);  
    }  
    out.flush();  
} catch (IOException e) { }  
}
```

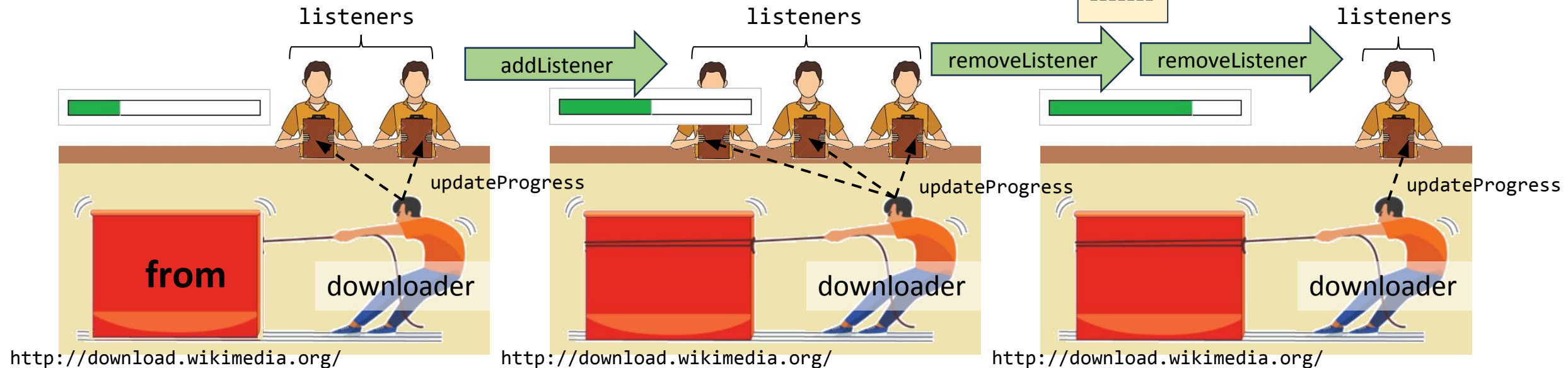
Lee del
servidor

Envía la notificación a
los listeners

Los peligros de los métodos “alienígenas”



<http://download.wikimedia.org/>



Los peligros de los métodos “alienígenas”



- Debido a que `addListener()`, `removeListener()` y `updateProgress()` están todos sincronizados [`synchronized`], varios hilos pueden llamarlos sin pisarse los pies unos de otros. Pero una trampa acecha en este código que podría llevar a un punto muerto [Deadlock] aunque solo haya un bloqueo en uso.
- El problema es que `updateProgress()` llama a un método alienígena [`onProgress(n)`], un método del que no sabe nada. Ese método podría hacer cualquier cosa, incluso adquirir otro cerrojo/candado [lock]. Si es así, entonces hemos adquirido dos bloqueos sin saber si lo hemos hecho en el orden correcto. Como acabamos de ver, eso puede llevar a un punto muerto [deadlock].
- La única solución es evitar llamar a métodos alienígenas mientras mantiene un cerrojo/candado. Una forma de lograr esto es hacer una copia defensiva de los oyentes antes de recorrerla:

Los peligros de los métodos “alienígenas”



```
private void updateProgress(int n) {  
    ArrayList<ProgressListener> listenersCopy;  
    synchronized(this) {  
➤      listenersCopy = (ArrayList<ProgressListener>)listeners.clone();  
    }  
    for (ProgressListener listener: listenersCopy)  
        listener.onProgress(n);  
}
```



- Este cambio mata varios pájaros de un tiro. No solo evita llamar a un método alienígena con un bloqueo retenido, sino que también minimiza el período durante el cual mantenemos el bloqueo. Mantener bloqueos durante más tiempo del necesario perjudica el rendimiento (al restringir el grado de simultaneidad alcanzable) y aumenta el peligro de un punto muerto. Este cambio también corrige otro error que no está relacionado con la simultaneidad: un oyente ahora puede llamar a `removeListener()` dentro de su método `onProgress()` sin modificar la copia de los oyentes que está en mitad de la iteración.

Resumen día 1

- Cubrimos cómo crear hilos y usar los bloqueos/locks intrínsecos integrados en cada Objeto de Java para hacer cumplir la exclusión mutua entre ellos. También vimos los tres los peligros principales de los subprocesos/hilos y bloqueos/cerrojos: condiciones de carrera, interbloqueo y memoria visibilidad, y discutimos algunas reglas que nos ayudan a evitarlas:
 - Sincronizar todo el acceso a las variables compartidas.
 - Tanto los hilos de escritura como los de lectura necesitan usar sincronización.
 - Adquirir múltiples cerrojos en un orden global fijo.
 - No llamar métodos “alienígenas” mientras mantiene un cerrojo/candado.
 - Mantenga los cerrojos durante el menor tiempo posible.

Autoestudio día 1

- Visite el sitio web del "modelo de memoria Java" de William Pugh.
 - <http://www.cs.umd.edu/~pugh/java/memoryModel/>
- Familiarícese con las preguntas frecuentes sobre JSR 133 (modelo de memoria Java).
 - <http://www.cs.umd.edu/~pugh/java/memoryModel/jsr-133-faq.html>
- ¿Qué garantías ofrece el modelo de memoria Java con respecto a la inicialización?
- ¿la seguridad? ¿Es siempre necesario utilizar bloqueos para publicar objetos de forma segura entre hilos?
- ¿Qué es el antipatrón de bloqueo con doble verificación [double-checked locking anti-pattern]? ¿Por qué es un anti-patrón?
 - <https://www.cs.umd.edu/~pugh/java/memoryModel/DoubleCheckedLocking.html>

Para hacer

- Experimente con el ejemplo original y roto de los “filósofos gastronómicos”. Intente modificar la cantidad de tiempo que los filósofos piensan y comen y el número de filósofos. ¿Qué efecto tiene esto en el tiempo que se tarda en llegar al punto muerto? Imagina que estás intentando depurar esto y quieres aumentar la probabilidad de reproducir el punto muerto, ¿qué harías?
- (Difícil) Cree un programa que demuestre que las escrituras en la memoria parecen estar reordenadas en ausencia de sincronización. Esto es difícil porque aunque el modelo de memoria de Java permite reordenar las cosas, la mayoría de los ejemplos simples no se optimizarán hasta el punto de demostrar realmente el problema.

Programas revisados

1.	Counting	Ok	
2.	CountingBette	NO	
3.	CountingFixed	Ok	
4.	DiningPhilosophers	Ok	
5.	DiningPhilosophersCondition	NO	
6.	DiningPhilosophersFixed	Ok	
7.	DiningPhilosophersTimeout	NO	
8.	EchoServer	NO	
9.	EchoServerBetter	NO	
10.	HelloWorld	Ok	
11.	HttpDownload	Ok	
12.	HttpDownloadBetter	NO	
13.	HttpDownloadFixed	Ok	
14.	Interruptible	NO	
15.	LinkedList	NO	
16.	Puzzle	Ok	
17.	Uninterruptible	NO	
18.	WordCount	NO	
19.	WordCountBatchConcurrentHashMap		NO
20.	WordCountConcurrentHashMap		NO
21.	WordCountProducerConsumer		NO
22.	WordCountSynchronizedHashMap		NO

Gracias